```
preemption state
rtdm_lock_irqsave(rtdm_lockctx_t context) : disable
preemption state
rtdm_lock_put(rtdm_lock_t lock) : release lock without
preemption restoration
rtdm_lock_put_irqrestore(rtdm_lock_t lock,rtdm_lockctx_t
context) : release lock and restore
preemption state.
rtdm_event_init(rtdm_event_t *event,unsigned long
pending): Initialize an event
void rtdm_signal_event(rtdm_event_t *event) : Signal an
event occurrence.
Int rtdm_event_wait(rtdm_event_t *event) : wait on event
occurrence
```

Clock services

Sometimes we need to get to know the uptime of the system, which the following API helps us do. The API returns the clock in nanoseconds. The resolution of this service depends on the system timer.

```
nanosecs_abs_t rtdm_clock_read(void) : Gets system time
```

Timer services

These are services for timer management which are provided by Xenomai project. There would APIs for a timer handler with which we start or stop.

```
int rtdm_timer_init(rtdm_timer_t *timer,rtdm_timer_
handler_t handler handler,const char *name) :
Initialize the timer
int rtdm_timer_start(rtdm_timer_t *timer,nanosecs_abs_t
expiry , nanosecs_abs_t interval , enum
rtdm_timer_mode mode) : start timer
```

User API in Xenomai

I am assuming readers are familiar with the device driver of Linux, as well as with *open*, *read*, *write*, *ioctl* and other system calls. Xenomai also supports its own system call so that the application calls the Xenomai kernel module. Some system calls supported by Linux are:

Open a device

```
int rt_dev_open(const char *path,int oflag,....)
```

Example:

```
int handle=rt_dev_open("omap-i2c",0);
```

Xenomai has an active device concept which means that *rt_dev_open()* calls the device by the name it was registered in the RTDM driver.

Read from device

```
ssize_t rt_dev_read(int fd,void *buf,size_t nbyte)
```

Example:

```
rt_dev_read(handle,(void *)buf,length)
```

Write to the device

```
ssize_t rt_dev_write ( int fd , const void * buf , size_t
nbyte )
```

Example:

```
rt_dev_write(handle,(void *)buf, size)
```

Issue an IOCTL

Example 1:

```
int rt_dev_ioctl(int fd,int request,....)
```

Example 2:

```
rt_dev_ioctl(device,command,value_to_pass)
```

Close device or socket

```
int rt_dev_close(int fd)
```

Example:

```
rt_dev_close(handle);
```

These are just a few APIs and examples of real-time driver development, shown for introductory purposes only. If you are keen, you will find more documentation on the Xenomai project site.

Some results

The I2C RTDM driver for AM335x Cortex-A8 developed by me was tested by finding the interrupt latency and scheduling latency. The interrupt and scheduling latency were less than 15 micro-seconds. Various load conditions, which were much less than the interrupt latency and scheduling latency in the Linux I2C driver, were found to be in milliseconds sometimes. Testing was done with the help of the *ftrace* tool. END

References

[1] *www.xenomai.org*

[2] *Building Embedded Linux Systems*, by Karim Yaghmour, Jon Master, Gilad Ben-Yossef and Philippe Gerum (second edition), O'Reilly

Acknowledgment

Babu Krishnamurthy, a freelancer and principal faculty at School of Embedded Software *(http://www.schoolofembedded.org/)*

By: Jay Kothari

The author is an embedded engineer and Linux kernel hacker who likes to play around with software. His interests include RTOS, Linux kernel, open source software, etc.You can contact him at *jaikothari10@gmail.com*